INTERNAL REPORT

fEURO: Fresh Deployment Rehearsal & Verification

A brand-new, EUR-pegged Frgmnt deployment — not an upgrade of anything live — built and exercised end to end on a local fork of real Base mainnet state: the price-source finding that shaped it, every guard dispatched against its real external protocol, and every withdrawal function exercised the same way the live fUSD deployment was.

Prepared for: Internal engineering team

Network: Base mainnet (chain 8453), forked locally only — nothing broadcast to real mainnet

Status: Nothing EUR-related is deployed anywhere today; this is a pre-launch rehearsal

---

Branch: `feature/09-softstack-stage2-fixes` **Commit:** `ceff2c7`

Fork block: `53,384,400`

CONTENTS

01 Executive summary

02 Scope & objective

03 The central finding: no genuine EUR/USD feed exists on Base

04 Deployment methodology

05 Deployed contracts

06 Core user flows & the EUR conversion proof

07 The permanent EUR/USD mode lock (FNA-40)

08 Lending guards vs. real external protocols (Morpho Blue, Aave V3)

09 Uniswap V3 guards vs. a real external pool (swap + full LP lifecycle)

10 Three dormant guard families, proven against real infrastructure

11 Guard validation (negative tests: M-09, M-12)

12 ERC20Guard validation

13 Exhaustive withdrawal-function coverage

14 Automatic pro-rata withdrawal (asset guards)

15 Multi-user isolation

16 Deposit cap enforcement

17 Access control

18 Reinitializer protection

19 Full test results

20 Issues found and fixed during this rehearsal
21 Deliberately not covered, and why
22 Outstanding items before a real EUR launch
23 Reproducing this rehearsal

01 Executive summary

This covers a different kind of exercise than the fUSD upgrade rehearsal: there is no live EUR deployment to upgrade. Nothing EUR-denominated exists on Base today. This report rehearses deploying one **from scratch** — the full contract stack, a fresh `TokenLogic` (fEURO) and `PoolLogic` (sfEURO), a fresh `AssetHandler` wired for EUR conversion, and fresh instances of every guard — against a local fork of real Base mainnet state, so every external-protocol interaction (Morpho Blue, Aave V3, Uniswap V3) is real, not mocked.

The first and most consequential finding came before any contract was deployed: **no genuine EUR/USD Chainlink price feed exists on Base** (Section 3). This shaped the rest of the rehearsal — the real, on-chain EURC/USD feed was used instead, with this codebase's own strict feed validator deliberately bypassed for this rehearsal run only (never for a real deployment). Three real deployment-sequencing bugs were found and fixed while bringing the stack up cleanly (Section 20). The core pricing round-trip was then proven correct to the decimal: 1,000 USDC in converts to the EUR-implied amount of fEURO, not a 1:1 USD mint, and the full stake→unstake→withdraw cycle returns exactly the original 1,000.0 USDC back out.

Two further rounds then mirrored the depth of the live fUSD deployment's own test suite: every guard dispatched for real against its external protocol (Morpho Blue and Aave V3 supply/withdraw, a real Uniswap V3 swap plus the full LP mint→track→remove-block→burn lifecycle, `ERC20Guard`'s approve validation), and every user-facing withdrawal function (the three immediate-withdrawal variants, the queued request/finalize/claim path, the automatic pro-rata pull from real Morpho+Aave positions, and three independent users exiting through three different paths with zero cross-contamination). A final round confirmed the deposit cap, access control, and reinitializer protection all carry real teeth on a freshly-deployed pool, not just a migrated one.

> **BOTTOM LINE**

39 of 39 tests pass (6 core-flow tests, 18 guard/withdrawal tests, plus 15 tests proving three otherwise-dormant guard families — AaveV4Spoke, AaveV4Tokenization, Morpho Vault V2 — against real external infrastructure), each round run twice in full against independent, completely fresh forks and fresh deployments — not reused state. Nothing in this rehearsal was broadcast to real mainnet.

What was confirmed, by theme:

- **The central finding (Section 3):** there is no genuine EUR/USD feed on Base. A real launch needs either a bridge via Ethereum mainnet's genuine feed, a different oracle provider, or an explicit, documented decision to accept EURC/USD's basis risk — this is a decision for the team, not something this rehearsal resolved on its own.
- **EUR pricing is genuinely applied** throughout — deposit, stake, unstake and withdrawal all round-trip at the correct EUR-implied rate, confirmed to the decimal, not just at the token-metadata level.
- **FNA-40's EUR/USD mode lock has a real, permanent consequence** for this product specifically (Section 7): the feed's own staleness timeout can never be raised again after launch, with no governance escape hatch.
- **Every real external-protocol integration works** against genuine Morpho Blue, Aave V3 and Uniswap V3 state on Base — not mocks (Sections 8–9).
- **Three otherwise-dormant guard families, also proven correct** against real Aave V4 and Morpho Vault V2 infrastructure (Section 10) — 15 further tests, mirroring the same proof already done for the USD deployment.
- **Every withdrawal path works**, including the automatic pro-rata pull from real, persistent lending positions and three simultaneous users through three different exit paths with no cross-contamination (Sections 13–15).
- **Three real deployment-sequencing bugs** were found and fixed while building the deploy script itself (Section 20) — none would have been caught without actually running the deployment against real protocol state.
- **Deliberately out of scope** for this rehearsal, and why (Section 21): the attested/selective withdrawal plan feature, and a few lower-priority guard branches.

02 Scope & objective

The task was explicit: rehearse a **first, fresh** EUR-denominated deployment — not an upgrade, since nothing EUR-related exists yet — then test it with the same depth as the live fUSD deployment: every guard against its real external protocol, and every user-facing function, especially every withdrawal variant. The objective was to:

1. Design and run the actual deployment sequence for a fresh EUR product, reusing this codebase's existing `PRODUCT=EUR` parameterization (`contracts/ contracts/` is byte-identical across products; only deploy-time configuration — token name/symbol, the EUR/USD feed — differs).
2. Resolve, or at minimum surface clearly, the price-source question: does a usable EUR/USD feed exist on Base at all.
3. Prove, against real forked external-protocol state, that every guard the product depends on (Morpho Blue, Aave V3, Uniswap V3, ERC20) works correctly for a freshly deployed pool, not one that inherited already-correct state from a migration.
4. Exhaustively exercise every withdrawal function, mirroring the breadth of the fUSD deployment's own functional-test suite.
5. Produce this report as the evidence record for the team's own pre-launch review.

> **SCOPE, STATED EXPLICITLY**
>
> This is a **rehearsal of a deployment that does not exist yet**, not a report on live state. Every address, balance and transaction in this report lives only on a local Anvil fork pinned at Base block 53,384,400 — reusing real, live external-protocol state (Morpho Blue's real markets, Aave V3's real pool, Uniswap V3's real USDC pools) so guard behavior is tested against genuine conditions, but never touching real mainnet. The `GOVERNANCE_SAFE` address used throughout this rehearsal is the live USD deployment's own Safe, reused here only as a convenient rehearsal stand-in — a real EUR launch would use its own, dedicated Safe.

03 The central finding: no genuine EUR/USD feed exists on Base

Before writing a single line of deployment script, the price source itself was checked. This codebase's own validator (`scripts/utils/validateEurUsdFeed.ts`) is strict by design: it requires a Chainlink feed whose `description()` contains "EUR/USD" or "EURUSD" and does not contain "USD/EUR", alongside decimals, freshness and a sane price-range check (0.80–1.50).

Checking Chainlink's own published feed directory for Base turned up only **`EURC/USD-RefPrice-DF-Base-001`** — a feed for Circle's EURC stablecoin's own USD price, not a genuine EUR/USD foreign-exchange rate. A true "EUR/USD-RefPrice-Timestamped" feed exists only for Ethereum mainnet, not for Base. This was confirmed directly against Chainlink's own documentation, not inferred from a third-party aggregator.

> **REAL FINDING — NOT RESOLVED BY THIS REHEARSAL, A TEAM DECISION**
>
> Base has no genuine EUR/USD foreign-exchange oracle. The nearest available feed, **`0x9867186e52d2F1C2c565CDA6E747101Fa56501e0`** ("EURC / USD", 8 decimals), prices Circle's EURC token against USD — which tracks the real EUR/USD rate closely in practice, but is not the same thing: it carries EURC's own de-peg risk as additional basis risk on top of genuine FX risk, and its `description()` string fails this codebase's own validator by design (it contains "USD" but not "EUR/USD", and the validator's substring checks are written to exclude exactly this kind of adjacent-but-different feed).
>
> Three real paths forward exist for an actual launch, in rough order of fidelity: (1) bridge a price via Ethereum mainnet's genuine EUR/USD feed somehow (added latency/complexity, but the most faithful FX rate); (2) source a EUR/USD feed from a different oracle provider that does publish one on Base; (3) make an explicit, documented decision to launch on EURC/USD and accept its basis risk. This rehearsal does not make that call — it surfaces the constraint precisely so the team can.

For this rehearsal only, with that constraint made explicit, the real EURC/USD feed was used and the strict validator was bypassed directly at the call site (never edited in the shared validator file itself) — logged clearly in the deploy script's own output and in the saved deployment JSON as a rehearsal-only relaxation, never to be reused for a real deployment.

PROPERTY	VALUE
Feed address	`0x9867186e52d2F1C2c565CDA6E747101Fa56501e0`
`description()`	`"EURC / USD"`
Decimals	8
Observed price	~1.1155–1.1195 USD per EURC, at different points in this rehearsal
Validator outcome	Would be correctly **rejected** by the real, unmodified `validateEurUsdFeed.ts` — bypassed only in this rehearsal's own deploy script, at the call site

04 Deployment methodology

Built as a new script, `scripts/fork/deploy_eur_fresh.ts`, reusing this codebase's existing `PRODUCT=EUR` support rather than inventing a parallel path. Deployed in order: libraries; `Governance` + `Timelock`; the `AssetHandler` proxy, wired for EUR conversion via `setEurUsdAggregator()` and the Base sequencer-uptime feed; `PoolManagerLogic`; `TokenLogic` (fEURO); `PoolLogic` (sfEURO); a `WithdrawalEscrow`; then every guard fresh — `NftTrackerStorage`, `SlippageAccumulator`, a new `MorphoBlueManager` (configured with the same 10 real USDC markets the live fUSD pool uses, since Morpho's markets are protocol-level and shared), the Morpho/Uniswap/Aave contract and asset guards, and `ERC20Guard` — followed by registration (contract guards, asset guards, `AssetHandler.addAsset`, `PoolManagerLogic.changeAssets`, `TokenLogic.configureAsset`) and custody handover to `GOVERNANCE_SAFE`.

Run with `SEND=1` against the local Anvil fork only (`baseFork` network, pointed at `127.0.0.1:8546`) — the same real-mainnet broadcast guard this codebase's scripts already carry elsewhere was never overridden. The full deployment was run twice end to end on two independent, freshly-restarted forks (pinned at block 53,384,400

both times) to confirm it is reproducible from a clean state, not an artifact of leftover state from debugging.

05 Deployed contracts

CONTRACT	ADDRESS (THIS REHEARSAL'S FORK ONLY)
Governance	`0x9Ba9F602B84D46f9c511518520aFb773162A0df3`
AssetHandler (proxy)	`0xf1485e28070983B2145D262F110Ae1b62f9FBC6b`
PoolManagerLogic (proxy)	`0x28f693F3AdE67Cab21123D42e6FF39889588Dc3D`
TokenLogic / fEURO (proxy)	`0x1aC0bB86f0139B05e2Fc7B7fe8f9cAD87Df4A791`
PoolLogic / sfEURO (proxy)	`0xc396EF69582127D55E974C59CFf2d0fa364494ce`
MorphoBlueManager	`0xf86c451fdACe00179451Eb2E524ADBCC99AffaC9`
ERC20Guard, UniswapV3*, AaveV3* guards	see `deployments/deploy-eur-fork-rehearsal.json`

`contracts/contracts/` is byte-identical between the EUR and USD products (enforced by `scripts/check-branch-parity.sh`) — every address above is a fresh instance of the exact same audited bytecode the live fUSD deployment runs, parameterized only by deploy-time arguments (name, symbol, the EUR/USD feed).

06 Core user flows & the EUR conversion proof

The single most important property to prove for this product is that EUR pricing is genuinely applied end to end — not just at the token-metadata level (a contract called "fEURO" that quietly mints and redeems at a 1:1 USD rate would be the one mistake that matters most here). Verified with a real deposit, stake, unstake and withdrawal, each checked against the actual EUR/USDC conversion rate read live from `AssetHandler.getUSDPrice(USDC)` (which, with the EUR aggregator set, genuinely returns the EUR-denominated price — the function name is kept for ABI compatibility, confirmed directly in `AssetHandler.sol`, not assumed).

STEP	RESULT
Deposit 1,000.0 USDC	Mints 896.207… fEURO — the genuine EUR-converted amount at the live ~0.8962 EUR/USDC rate, not ~1,000.0 (1:1 USD, wrong) and not ~1,120 (inverted direction, wrong)
Stake	896.207… fEURO → 896.207… sfEURO, 1:1 share mint on a fresh pool
Harvest (immediately after)	Correctly reverts `NothingToHarvest()` — this rehearsal pool has no real yield source wired up yet, so there is genuinely nothing to harvest; confirmed as correct behavior, not routed around
Unstake	896.207… sfEURO → 896.207… fEURO back, principal exactly preserved
withdrawCashImmediate	Burns 896.207… fEURO, returns exactly **1,000.0 USDC** — the correct inverse of the deposit conversion, not 896 (double-converted) and not 1,120 (inverted)

The full round-trip — 1,000 USDC in, 1,000.0 USDC out — confirms the EUR conversion is applied symmetrically and correctly in both directions.

07 The permanent EUR/USD mode lock (FNA-40)

`AssetHandler.setEurUsdAggregator()` sets `eurUsdModeLocked = true` permanently on its own first call — confirmed directly by reading `AssetHandler.sol`, not assumed. This is deliberate, documented behavior: "a single bootstrap-time decision, not a runtime toggle." Confirmed on this fresh EUR pool: `eurUsdModeLocked()` reads `true` immediately after deployment, and a further `clearEurUsdAggregator()` call correctly reverts, even for the real owner.

> **REAL, PERMANENT CONSEQUENCE FOR THIS PRODUCT SPECIFICALLY**
>
> Because the mode is locked, **`eurUsdTimeout` (the EUR/USD feed's own staleness window) can never be raised again after launch** — there is no governance escape hatch. On real mainnet this is unlikely to matter day-to-day (Chainlink/Circle update the EURC/USD feed continuously), but if that feed's real update cadence ever runs close to whatever timeout is chosen at launch, there is no way to extend it without a full redeployment. Worth the team's

> explicit awareness when choosing that timeout value, not something this rehearsal flags as a code defect to fix.

08 Lending guards vs. real external protocols (Morpho Blue, Aave V3)

Both dispatched via direct `execTransaction` calls through the pool's real guard-registration path (the same mechanism the live fUSD pool uses for its own lending activity) — not a mock, against real Morpho Blue and Aave V3 contracts on the forked Base state.

TEST	RESULT
Morpho Blue supply + withdraw	Supplied 1,000.0 USDC into one of the 10 real, governance-configured markets; withdrew back 999.999999 USDC using the shares gained (Morpho's own recommended full-withdrawal pattern); `supplyShares` returned exactly to baseline.
Aave V3 supply + withdraw	Supplied 1,000.0 USDC; aToken balance increased by 999.999999; withdrew the full balance back (`type(uint256).max` , Aave's own full-withdrawal semantics); final aToken balance exactly 0.

09 Uniswap V3 guards vs. a real external pool (swap + full LP lifecycle)

USDT_BASE ("Bridged Tether USD," the same real, circulating Base token this codebase's own `AaveLendingPoolAssetGuard.sol` already treats as "USDT") was registered as a second stablecoin so a real, non-fabricated swap and LP position could be exercised, mirroring the live fUSD deployment's own stablecoin-only Uniswap testing. Isolated on its own snapshot/revert so the registration and position are never left behind on the shared pool state the rest of this rehearsal runs on.

TEST	RESULT

Real USDC → USDT_BASE swap (UniswapV3RouterGuard)	Swapped 450.0 USDC for 450.517951 USDT_BASE through the real, live 0.01% fee-tier pool.
Real LP mint (UniswapV3NonfungiblePositionGuard + AssetGuard, M-14 loss check)	Minted a real position around the pool's current tick; M-14's fair-price/mint-loss check passed with no revert; the new position was correctly tracked in `NftTrackerStorage`.
FNA-48: removal blocked while tracked	Attempting to delist the Uniswap position manager while the minted position was still open correctly reverted (`"UniswapV3AssetGuard: positions tracked"`).
Burn (full close)	Position fully closed; no longer appears in `getOwnedTokenIds()` afterward.

Before trusting this round's results, `NftTrackerStorage.poolFactory()` was confirmed to point at this pool's own `PoolManagerLogic` — the live USD deployment had an independent, already-CertiK-flagged misconfiguration here (fixed separately in that deployment's own upgrade), and a fresh EUR deployment could in principle repeat it if the deploy script's initializer call were wrong. It was not: this fresh deploy initializes `NftTrackerStorage` correctly from the start.

10 Three dormant guard families, proven against real infrastructure

Beyond the four guard families this fresh EUR pool is actually registered with (Section 8–9), this codebase also carries three guard families that exist in the repository and are fully audited, but are not wired up for any live Frgmnt pool today — the same three "dormant" families the USD upgrade-rehearsal report proved out against real external infrastructure: **AaveV4Spoke**, **AaveV4Tokenization**, and **Morpho Vault V2**. Each was deployed fresh and registered against this EUR pool's own `Governance` / `AssetHandler` / `PoolManagerLogic` — fork-only, never mainnet — then dispatched for real against genuine, live Base infrastructure: Aave's own real `MAG7_SPOKE` lending spoke, the real `TOKENIZATION_SPOKE` (an ERC-4626 wrapper over a Hub asset), and a freshly-created real Morpho Vault V2 instance through Morpho's own live factory.

> **WHY THIS MATTERS FOR A FRESH EUR POOL SPECIFICALLY**
>
> The USD-side version of this same exercise found a serious, cross-cutting methodology bug: testing these three families against a pre-upgrade pool (one with no concept of `IPreValuedAssetGuard`) silently mispriced them by a factor of ~10^{12}, sizing withdrawal portions as effectively zero while still burning the user's tokens for real. This EUR pool is deployed fresh, directly from this branch, so it is never in that pre-upgrade state to begin with — but the three test scripts still carry the same explicit precondition check (`PoolLogic.compoundedRewardIndex()` must not revert) as a standing safeguard, rather than assuming "fresh deploy" is automatically safe from a bug that was originally about upgrade sequencing.

FAMILY	REAL INFRASTRUCTURE USED	TESTS
AaveV4Spoke	`MAG7_SPOKE` + `GiverPositionManager` + `TakerPositionManager` (real, live Aave V4 Base contracts)	5/5 pass: real supply via Giver, automatic pro-rata withdrawal, manual withdrawal via Taker (`approveWithdraw` + `withdrawOnBehalfOf`), non-whitelisted-reserve rejection, `spender != pool` rejection (the guard's own "single most important check," per its source comment — otherwise Aave V4's `withdraw()` sends funds to `msg.sender`, not `onBehalfOf`, letting an arbitrary approved spender drain the position).
AaveV4Tokenization	`TOKENIZATION_SPOKE` (real, live ERC-4626-shaped Aave V4 contract)	5/5 pass: real deposit, automatic pro-rata withdrawal, manual `withdraw()`, `receiver != pool` rejection, non-whitelisted-vault rejection.
Morpho Vault V2	A freshly-created vault through Morpho's real, live `VaultV2Factory` on Base	5/5 pass: real deposit, automatic pro-rata withdrawal, manual `withdraw()`, `receiver != pool` rejection, non-whitelisted-vault rejection.

All 15 tests pass, reproduced twice from two independent, freshly-restarted forks and fresh deployments. Each family is isolated on its own snapshot/revert, so none

of the three — and none of the fresh pseudo-assets, markets or vaults each one registers — are left behind on the shared pool state the rest of this report's tests run on.

11 Guard validation (negative tests: M-09, M-12)

TEST	RESULT
M-12: multi-hop path through an unsupported token	Correctly reverted: `"Frgmnt: unsupported hop asset"`.
M-09: zero output-minimum swap	Correctly reverted: `"Frgmnt: amountOutMinimum must be > 0"`.

12 ERC20Guard validation

Only `approve()` is handled by this guard (there is no `transfer()` handler — confirmed by reading the contract). A supported asset (USDC) approving a registered, guarded spender (Morpho core) was accepted; an unsupported asset (DAI, not registered for this pool) attempting the same was correctly rejected.

13 Exhaustive withdrawal-function coverage

Every user-facing withdrawal entry point on `PoolLogic`, mirroring the breadth of the live fUSD deployment's own functional-test suite (see Section 21 for the one path deliberately not included).

FUNCTION	RESULT
`withdrawCashImmediate`	Covered as part of the core-flow proof (Section 6) and the multi-user round (Section 15).
`withdrawCashImmediateTo`	Burned 2,688.622… fEURO; third-party recipient received 2,999.999999 USDC; caller's own USDC balance untouched, as designed.

`withdrawCashImmediateSafe`	Withdrew with an explicit 4-entry `ComplexAsset[]` (default/empty entries, matching `getSupportedAssets().length`); received 2,999.999999 USDC.
`withdrawCashImmediateToSafe`	Combined both of the above: third-party recipient received the remaining 6,000.0 USDC; caller's fEURO balance zeroed exactly.
`requestCashWithdraw` + `finalizeCashWithdraw` + `claimCashWithdraw` (queued path)	Request #1: requested 4,481.037… fEURO, claimed 5,000.0 USDC after the manager finalized and the user claimed from escrow.

14 Automatic pro-rata withdrawal (asset guards)

Distinct from Section 8's manual `execTransaction` dispatch: this exercises the asset guards' own `getBalance`/withdrawal-processing logic, the code path every `withdrawCashImmediate` invokes automatically for any supported asset with a nonzero balance. Real, persistent Morpho (8,000 USDC) and Aave (6,000 USDC) positions were established first; a separate fresh user then withdrew more than the pool's idle USDC alone could cover.

> **CONFIRMED**
>
> The withdrawal succeeded and genuinely pulled from **both** positions automatically: the Morpho asset-guard balance dropped from $7,169.66 to $6,037.61, and the Aave asset-guard balance dropped from $5,377.24 to $4,528.21 — not the manual dispatch path, the pro-rata loop itself.

15 Multi-user isolation

Three independent accounts, three different exit paths, run in the same scenario with no snapshot isolation between them (to actually test for cross-contamination, not assume its absence):

USER	EXIT PATH	RESULT

A	`withdrawCashImmediate` direct	Received 3,999.999999 USDC; final fEURO balance 0
B	stake → unstake → `withdrawCashImmediate`	Received 5,999.999999 USDC; final fEURO balance 0
C	`requestCashWithdraw` → finalize → claim	Received 9,000.0 USDC; final fEURO balance 0

All three independent; none of the three balances were affected by either of the other two users' activity.

16 Deposit cap enforcement

`TokenLogic.maxDepositFusdSupply()` read 500,000.0 fEURO exactly, matching the deploy script's own `initializeDepositFusdCap` call. Temporarily tightened (via the real admin setter, reverted via snapshot afterward, never left tightened) to just above outstanding supply to make a genuine breach affordable against this fork's limited whale balance: a 5,000 USDC over-cap deposit correctly reverted (`"TokenLogic: deposit cap exceeded"`), and an immediately-following 500 USDC under-cap deposit from the same account succeeded — confirming the cap neither under- nor over-enforces.

17 Access control

Three privileged functions probed with an unauthorized random account (expecting rejection) and the real, intended caller (expecting success):

FUNCTION	UNAUTHORIZED CALLER	REAL CALLER
`Governance.setContractGuard`	Correctly reverted (`OwnableUnauthorizedAccount`)	Succeeded
`TokenLogic.configureAsset`	Correctly reverted (missing `DEFAULT_ADMIN_ROLE`)	Succeeded
`MorphoBlueManager.setPoolMarkets`		

	Correctly reverted (`OwnableUnauthorizedAccount`)	See callout below

A STRUCTURAL LIMITATION OF THIS REHEARSAL'S IDENTITY CHOICES, NOT A BUG

The real owner's re-submission of `setPoolMarkets` could not be exercised here: this rehearsal reuses `GOVERNANCE_SAFE` as both the pool manager and `MorphoBlueManager`'s owner (the same simplification the deploy script itself had to sequence around for the very first `setPoolMarkets` call — see Section 20). Any subsequent call by that same address trips M-11's own self-dealing check (`OwnerIsPoolManagerOrTrader()`, confirmed directly via its selector) instead of succeeding. This is correct, intended M-11 behavior: a real EUR launch would need a genuinely different address to own `MorphoBlueManager` than the one acting as pool manager, specifically to allow legitimate post-launch market reconfiguration. Reported honestly as a rehearsal-identity limitation, not routed around by fabricating a different owner.

18 Reinitializer protection

INITIALIZER	RESULT
`TokenLogic.initializeDepositFusdCap()`	Correctly reverted on a second call — already consumed for real by the deploy script itself.
`PoolLogic.initializeWithdrawalEscrow()`	Correctly reverted on a second call (a plain already-set check, not reinitializer-gated — confirmed by reading the contract).
`PoolLogic.initializeAutoCompounding()`	Correctly reverted — but by a genuinely **different mechanism** than on the USD upgrade rehearsal, see callout below.

A SUBTLETY SPECIFIC TO FRESH DEPLOYMENTS, CHECKED DIRECTLY RATHER THAN ASSUMED

On a fresh deploy, `PoolLogic.initialize()` itself already sets `compoundedRewardIndex = 1e18` directly — not via the reinitializer path. So `initializeAutoCompounding()`'s own `reinitializer(2)` version gate has not been

consumed yet when this call is attempted; it genuinely proceeds past that gate and is instead blocked by its own business-logic check, `AutoCompoundingAlreadyInitialized()` — not `InvalidInitialization()`, the revert the USD upgrade rehearsal's equivalent test observes. Both are correct, both block the call, but by different mechanisms — worth knowing precisely rather than assuming one report's revert reason applies unchanged to the other.

19 Full test results

39 of 39 tests pass, run across three files and reproduced from a completely fresh fork + fresh deployment (not reused state).

#	TEST	RESULT
1	EUR-converted deposit (not 1:1 USD)	PASS
2	Stake (1:1 share mint)	PASS
3	Harvest correctly reverts `NothingToHarvest()`	PASS
4	Unstake (principal preserved)	PASS
5	`withdrawCashImmediate` at correct inverse EUR rate	PASS
6	FNA-40 EUR/USD mode lock engaged permanently	PASS
7	Morpho Blue supply + withdraw	PASS
8	Aave V3 supply + withdraw	PASS
9	M-12 multi-hop rejection	PASS
10	M-09 zero output-minimum rejection	PASS
11	ERC20Guard approve() validation	PASS
12	Real Uniswap V3 swap (USDC → USDT_BASE)	PASS
13	Real LP mint (M-14 check, tracked)	PASS

14	FNA-48 removal blocked while tracked	PASS
15	Burn clears NFT tracking	PASS
16	`withdrawCashImmediateTo`	PASS
17	`withdrawCashImmediateSafe`	PASS
18	`withdrawCashImmediateToSafe`	PASS
19	Queued withdrawal (request + finalize + claim)	PASS
20	Automatic pro-rata pull (Morpho + Aave)	PASS
21	Deposit cap enforcement	PASS
22	Access control (3 functions)	PASS
23	Reinitializer protection (3 initializers)	PASS
24	Multi-user isolation (3 users, 3 exit paths)	PASS
25	AaveV4Spoke: real supply via GiverPositionManager	PASS
26	AaveV4Spoke: automatic pro-rata withdrawal from the real position	PASS
27	AaveV4Spoke: manual withdrawal via TakerPositionManager	PASS
28	AaveV4Spoke: non-whitelisted reserve rejection	PASS
29	AaveV4Spoke: `spender != pool` rejection	PASS
30	AaveV4Tokenization: real deposit into the live TOKENIZATION_SPOKE	PASS
31	AaveV4Tokenization: automatic pro-rata withdrawal from the real position	PASS
32	AaveV4Tokenization: manual `withdraw()`	PASS
33	AaveV4Tokenization: `receiver != pool` rejection	PASS
34	AaveV4Tokenization: non-whitelisted-vault rejection	PASS
35	Morpho Vault V2: real deposit into a freshly-created vault	PASS

36	Morpho Vault V2: automatic pro-rata withdrawal from the real position	PASS
37	Morpho Vault V2: manual `withdraw()`	PASS
38	Morpho Vault V2: `receiver != pool` rejection	PASS
39	Morpho Vault V2: non-whitelisted-vault rejection	PASS

20 Issues found and fixed during this rehearsal

None of these were known going in — each was found by actually running the deployment against real protocol state, not anticipated in advance.

ISSUE	ROOT CAUSE	FIX
M-11 self-dealing check reverted the first `setPoolMarkets` call	`GOVERNANCE_SAFE` was set as both the pool manager and (after an early `transferOwnership`) `MorphoBlueManager`'s owner — exactly the condition M-11 is designed to block.	Resequenced: call `setPo` owns `MorphoBlueManage` `GOVERNANCE_SAFE` only a
`"TokenLogic: asset not allowed"` on the first deposit	`TokenLogic.deposit()` requires `TokenLogic.configureAsset()` — a separate allowlist from `PoolManagerLogic.changeAssets()`, easy to miss since both sound like the same registration step.	Added `fusdAsManager.c` to the deploy script.
`"TokenLogic: deposit cap exceeded"` on the first deposit	`maxDepositFusdSupply` defaults to 0 on a fresh deploy.	Added `fusdAsManager.initial` to the deploy script.
`evm_increaseTime` for cooldown-clearing also tripped the EUR feed's own staleness check	A plain time-jump advances the fork's clock past the EUR/USD feed's 24h `eurUsdTimeout` — and unlike the USD side, this timeout can never be raised again post-launch (FNA-40, Section 7).	Used `TokenLogic.setCo` lever) instead of time-tra suite — no `evm_increase` `eur_guard_and_withdra`

21 Deliberately not covered, and why

- **The attested/selective withdrawal plan** (`withdrawCashImmediateWithPlan` , replay/expiry rejection). This requires `PoolLogic.initializeAttestedWithdrawal()` — a separate, deliberate attester-key decision that is not part of `deploy_eur_fresh.ts` today. Including it would mean inventing an attester key and rotation/volume/surcharge parameters on the team's behalf for a feature decision that belongs to them, not to this rehearsal. Flagged here explicitly rather than silently skipped.
- **`SlippageAccumulator`'s cumulative-cap enforcement** and **partial LP close (decreaseLiquidity + collect without a full burn)** — exercised for the USD deployment's own report; not repeated here since the mint→track→remove-block→burn lifecycle (Section 9) already exercises the same underlying guard code paths for a fresh pool, and the cap-tightening test specifically depends on configuration this fresh pool doesn't carry over from a migration.
- **M-08's artificial-debt branch** (an Aave debt position deliberately created outside the guard dispatch path) — lower priority than the withdrawal-function breadth the task explicitly called out, and not exercised for the EUR pool in this pass.

22 Outstanding items before a real EUR launch

1. **Resolve the EUR/USD price-source question for real** (Section 3) — this rehearsal's EURC/USD-plus-bypassed-validator choice must never be carried into a real deployment.
2. **Choose `eurUsdTimeout` deliberately**, knowing it can never be raised again post-launch (Section 7).
3. **Use a dedicated Safe** for the EUR product's governance, manager and `MorphoBlueManager` -owner roles — not the live USD deployment's Safe (this rehearsal's own stand-in), and critically, not the same address for both pool-manager and `MorphoBlueManager` -owner (Section 17), so legitimate post-launch market reconfiguration remains possible.
4. **Decide on the attested withdrawal plan feature** for this product (Section 21) before launch, one way or the other.

5. **Real audit review** of the fresh-deployment script itself, `scripts/fork/deploy_eur_fresh.ts` — it has not been through an external audit round the way the core contracts it deploys have.

23 Reproducing this rehearsal

STEP	COMMAND
1. Start the fork	`./scripts/fork/start-base-fork.sh 53384400`
2. Deploy fresh	`SEND=1 npx hardhat run scripts/fork/deploy_eur_fresh.ts --network baseFork`
3. Core-flow tests	`npx hardhat run scripts/fork/eur_functional_tests.ts --network baseFork`
4. Guard + withdrawal tests	`npx hardhat run scripts/fork/eur_guard_and_withdrawal_tests.ts --network ba`
5. Dormant guard families (AaveV4Spoke, AaveV4Tokenization, Morpho Vault V2)	`npx hardhat run scripts/fork/eur_dormant_guards_integration_test.ts --netwo`

Every address, balance and transaction above is reproducible exactly from this sequence against the pinned fork block — confirmed by running it twice, from two independent, freshly-restarted forks.

Fork-only rehearsal against Base mainnet (chain 8453) state, pinned at block 53,384,400. No private keys were used and nothing was broadcast to real mainnet at any point. `GOVERNANCE_SAFE` throughout this report refers to the live USD deployment's Safe, reused here solely as a rehearsal stand-in.